FioFora — Projeto Integrado (Software) · Sprint 6
Membro 3 — Antonio de Azambuja Fehring

Cálculo de severidade e desconto no model Peça

O desconto do FioFora sai de uma conta, não de um chute do vendedor. Essa é a diferença entre o nosso site e um brechó. Até agora a regra só existia escrita na Modelagem de Dados da Sprint 4. Esta entrega põe ela em código.

São três arquivos, todos em `catalogo/`:

Arquivo	O que tem dentro
`severidade.py`	A conta: nota, faixa, desconto e preço final.
`models.py`	Os métodos da Peça e do Defeito que chamam a conta.
`tests.py`	16 testes.

A regra

O defeito é classificado em três critérios, cada um valendo de 1 a 3. O tipo (estético, funcional reparável, funcional não reparável), a localização (escondido, semivisível, visível) e a extensão (até 2 cm, de 2 a 10 cm, acima de 10 cm). Os pesos vieram da calibração feita com as oito entrevistas da Sprint 2:

```
nota = 0,45 × tipo + 0,35 × localização + 0,20 × extensão
```

A nota vai de 1,00 a 3,00. Até 1,70 a peça é Leve, até 2,30 é Moderada, acima disso é Alta. O desconto cresce junto com a nota, de 15% a 50%, e é arredondado para o múltiplo de 5% mais próximo. Preço quebrado não convence ninguém.

Decisões

Quando a peça tem mais de um defeito, vale o mais grave, e não a média dos dois. Uma camisa com um furo visível e um fio solto continua sendo uma camisa com um furo visível. Se a conta tirasse média, acrescentar um defeito pequeno deixaria a peça parecer melhor do que ela é, que é exatamente o contrário do que o site promete.

Nota, faixa, desconto e preço final não são colunas no banco. São calculados toda vez que a peça é lida. Se o desconto fosse coluna, bastava recalibrar um peso depois de uma nova rodada de entrevistas para que todas as peças já cadastradas passassem a mostrar um número que não corresponde mais à regra, e não haveria como saber quais estavam desatualizadas.

A conta está num arquivo só. A vitrine, a página do produto e o simulador chamam a mesma função em vez de cada um fazer a sua. O requisito SIM-07 pede que os três mostrem o mesmo número, e isso não para de pé se a fórmula estiver copiada em três lugares.

Resultado

Essas são as 27 combinações possíveis, calculadas pelo código sobre uma peça de R$ 289,90:

Critérios	Nota	Faixa	Desc.	Preço
1·1·1	1.00	Leve	15%	246.41
1·1·2	1.20	Leve	20%	231.92
1·2·1	1.35	Leve	20%	231.92
1·1·3	1.40	Leve	20%	231.92
2·1·1	1.45	Leve	25%	217.42
1·2·2	1.55	Leve	25%	217.42
2·1·2	1.65	Leve	25%	217.42
1·3·1	1.70	Leve	25%	217.42
1·2·3	1.75	Moderada	30%	202.93
2·2·1	1.80	Moderada	30%	202.93
2·1·3	1.85	Moderada	30%	202.93
1·3·2	1.90	Moderada	30%	202.93
3·1·1	1.90	Moderada	30%	202.93
2·2·2	2.00	Moderada	35%	188.43

Critérios	Nota	Faixa	Desc.	Preço
1·3·3	2.10	Moderada	35%	188.43
3·1·2	2.10	Moderada	35%	188.43
2·3·1	2.15	Moderada	35%	188.43
2·2·3	2.20	Moderada	35%	188.43
3·2·1	2.25	Moderada	35%	188.43
3·1·3	2.30	Moderada	40%	173.94
2·3·2	2.35	Alta	40%	173.94
3·2·2	2.45	Alta	40%	173.94
2·3·3	2.55	Alta	40%	173.94
3·3·1	2.60	Alta	45%	159.44
3·2·3	2.65	Alta	45%	159.44
3·3·2	2.80	Alta	45%	159.44
3·3·3	3.00	Alta	50%	144.95

O desconto nunca cai quando a nota sobe, nunca fica abaixo de 15%, nunca passa de 50% e é sempre múltiplo de 5%. Os testes conferem isso nas 27.

Testes

```
$ python manage.py test catalogo

Found 16 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
................
----------------------------------------------------------------------
Ran 16 tests in 0.018s

OK
```

Como as outras telas usam

Quem for fazer a vitrine, a página do produto ou o simulador não precisa saber a fórmula. Basta ler as propriedades da peça:

```
peca.nota_severidade      # Decimal("2.35")
peca.faixa_severidade     # "Alta"
peca.desconto_percentual  # 40
peca.preco_final          # Decimal("173.94")
```

Um aviso: a conta usa `Decimal`, não `float`. Em float, `0.45 * 3` dá `1.3499999999999999`, e o arredondamento para 5% escorrega em alguns casos. Se alguém converter para float no meio do caminho, o desconto muda.

catalogo/severidade.py

```
"""Cálculo de severidade e desconto do FioFora.

Regra RN-06, com os pesos da Modelagem de Dados (Sprint 4).

Não importa nada do Django de propósito. Recebe qualquer objeto que tenha
tipo, localizacao e extensao, então a mesma função serve o model, o simulador
e os testes.
"""
from dataclasses import dataclass
from decimal import ROUND_HALF_UP, Decimal

PESO_TIPO = Decimal("0.45")
PESO_LOCAL = Decimal("0.35")
PESO_EXTENSAO = Decimal("0.20")

DESCONTO_MIN = Decimal("0.15")
DESCONTO_MAX = Decimal("0.50")
PASSO_DESCONTO = Decimal("0.05")

LIMITE_LEVE = Decimal("1.70")
LIMITE_MODERADA = Decimal("2.30")

LEVE, MODERADA, ALTA = "Leve", "Moderada", "Alta"


@dataclass(frozen=True)
class Parametros:
    """Uma versão dos pesos. Recalibrar é criar outra, não editar esta."""

    peso_tipo: Decimal = PESO_TIPO
    peso_local: Decimal = PESO_LOCAL
    peso_extensao: Decimal = PESO_EXTENSAO
    desconto_min: Decimal = DESCONTO_MIN
    desconto_max: Decimal = DESCONTO_MAX


VIGENTE = Parametros()


@dataclass(frozen=True)
class Classificacao:
    nota: Decimal
    faixa: str
    desconto: Decimal          # fração: 0.25 é 25%
    preco_final: Decimal

    @property
    def desconto_percentual(self):
        return int(self.desconto * 100)


def nota_do_defeito(defeito, prm=VIGENTE):
    """Nota de um defeito, de 1,00 a 3,00."""
    return (
        prm.peso_tipo * Decimal(defeito.tipo)
        + prm.peso_local * Decimal(defeito.localizacao)
        + prm.peso_extensao * Decimal(defeito.extensao)
    )


def faixa_da_nota(nota):
    if nota <= LIMITE_LEVE:
        return LEVE
    if nota <= LIMITE_MODERADA:
```

```
        return MODERADA
    return ALTA


def desconto_da_nota(nota, prm=VIGENTE):
    """Desconto da nota, como fração arredondada a múltiplos de 5%."""
    bruto = prm.desconto_min + ((nota - 1) / 2) * (prm.desconto_max - prm.desconto_min)
    passos = (bruto / PASSO_DESCONTO).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    return min(passos * PASSO_DESCONTO, prm.desconto_max)


def classificar(defeitos, preco_original, prm=VIGENTE):
    """Classifica a peça pelos seus defeitos.

    Com mais de um defeito vale o mais grave, não a média.
    Peça sem defeito não entra na vitrine, então devolve None.
    """
    defeitos = list(defeitos)
    if not defeitos:
        return None

    nota = max(nota_do_defeito(d, prm) for d in defeitos)
    desconto = desconto_da_nota(nota, prm)
    preco_final = (Decimal(preco_original) * (1 - desconto)).quantize(
        Decimal("0.01"), rounding=ROUND_HALF_UP
    )

    return Classificacao(
        nota=nota.quantize(Decimal("0.01")),
        faixa=faixa_da_nota(nota),
        desconto=desconto,
        preco_final=preco_final,
    )
```

catalogo/models.py

```
"""Models do catálogo.

A entrega desta sprint são os métodos de classificação da Peça, no bloco
marcado mais abaixo. Peca e Defeito estão aqui no formato da Modelagem de
Dados para que os métodos rodem e possam ser testados.
"""
from decimal import Decimal

from django.core.validators import MaxValueValidator, MinValueValidator
from django.db import models

from . import severidade


class Peca(models.Model):
    CATEGORIAS = [
        ("camisa", "Camisa"),
        ("calca", "Calça"),
        ("vestido", "Vestido"),
        ("casaco", "Casaco"),
        ("acessorio", "Acessório"),
    ]
    STATUS = [
        ("disponivel", "Disponível"),
        ("reservada", "Reservada"),
        ("vendida", "Vendida"),
        ("pausada", "Pausada"),
    ]
```

```
    titulo = models.CharField("título", max_length=120)
    categoria = models.CharField(max_length=20, choices=CATEGORIAS)
    tamanho = models.CharField(max_length=10)
    cor = models.CharField(max_length=40, blank=True)
    composicao = models.CharField("composição", max_length=120, blank=True)
    preco_original = models.DecimalField("preço original", max_digits=9, decimal_places=2)
    descricao = models.TextField("descrição", blank=True)
    status = models.CharField(max_length=12, choices=STATUS, default="disponivel")
    publicada_em = models.DateField(auto_now_add=True)

    class Meta:
        verbose_name = "peça"
        verbose_name_plural = "peças"

    def __str__(self):
        return self.titulo

    # ---------------- classificação: severidade e desconto ----------------
    # Nenhum destes valores é coluna no banco. São calculados na leitura.

    def classificacao(self):
        return severidade.classificar(self.defeitos.all(), self.preco_original)

    @property
    def nota_severidade(self):
        """De 1,00 a 3,00. Com mais de um defeito, vale o mais grave."""
        c = self.classificacao()
        return c.nota if c else None

    @property
    def faixa_severidade(self):
        """Leve, Moderada ou Alta. É o texto do selo na vitrine."""
        c = self.classificacao()
        return c.faixa if c else None

    @property
    def desconto_percentual(self):
        """Inteiro múltiplo de 5. 25 significa 25%."""
        c = self.classificacao()
        return c.desconto_percentual if c else None

    @property
    def preco_final(self):
        c = self.classificacao()
        return c.preco_final if c else self.preco_original


class Defeito(models.Model):
    TIPOS = [
        (1, "Estético"),
        (2, "Funcional reparável"),
        (3, "Funcional não reparável"),
    ]
    LOCALIZACOES = [
        (1, "Escondido"),
        (2, "Semivisível"),
        (3, "Visível"),
    ]
    EXTENSOES = [
        (1, "Até 2 cm"),
        (2, "De 2 a 10 cm"),
        (3, "Acima de 10 cm"),
    ]
    CRITERIO = [MinValueValidator(1), MaxValueValidator(3)]
```

```
    peca = models.ForeignKey(Peca, related_name="defeitos", on_delete=models.CASCADE)
    tipo = models.PositiveSmallIntegerField(choices=TIPOS, validators=CRITERIO)
    localizacao = models.PositiveSmallIntegerField(
        "localização", choices=LOCALIZACOES, validators=CRITERIO
    )
    extensao = models.PositiveSmallIntegerField(
        "extensão", choices=EXTENSOES, validators=CRITERIO
    )
    medida_cm = models.DecimalField(
        "medida (cm)", max_digits=5, decimal_places=1, null=True, blank=True
    )
    visivel_vestida = models.BooleanField("visível com a peça vestida", default=False)
    reparavel = models.BooleanField("reparável", default=False)
    descricao = models.TextField("descrição", blank=True)

    def __str__(self):
        return f"{self.get_tipo_display()} · {self.get_localizacao_display()}"

    @property
    def nota(self):
        """Nota deste defeito sozinho, para a ficha do produto."""
        return severidade.nota_do_defeito(self).quantize(Decimal("0.01"))
```

catalogo/tests.py

```
"""Testes do cálculo de severidade e desconto — FioFora."""
from decimal import Decimal

from django.test import TestCase

from .models import Defeito, Peca
from .severidade import classificar, desconto_da_nota, faixa_da_nota


class Falso:
    """Um defeito qualquer, sem passar pelo banco."""

    def __init__(self, tipo, localizacao, extensao):
        self.tipo, self.localizacao, self.extensao = tipo, localizacao, extensao


class CalculoDaNota(TestCase):
    def test_peca_menos_grave_possivel(self):
        c = classificar([Falso(1, 1, 1)], 200)
        self.assertEqual(c.nota, Decimal("1.00"))
        self.assertEqual(c.faixa, "Leve")
        self.assertEqual(c.desconto_percentual, 15)
        self.assertEqual(c.preco_final, Decimal("170.00"))

    def test_peca_mais_grave_possivel(self):
        c = classificar([Falso(3, 3, 3)], 200)
        self.assertEqual(c.nota, Decimal("3.00"))
        self.assertEqual(c.faixa, "Alta")
        self.assertEqual(c.desconto_percentual, 50)
        self.assertEqual(c.preco_final, Decimal("100.00"))

    def test_tipo_pesa_mais_que_extensao(self):
        """Um defeito funcional pequeno é pior que um estético grande."""
        funcional_pequeno = classificar([Falso(3, 1, 1)], 100).nota
        estetico_grande = classificar([Falso(1, 1, 3)], 100).nota
        self.assertGreater(funcional_pequeno, estetico_grande)

    def test_nota_nunca_sai_do_intervalo(self):
```

```
        for t in (1, 2, 3):
            for l in (1, 2, 3):
                for e in (1, 2, 3):
                    nota = classificar([Falso(t, l, e)], 100).nota
                    self.assertGreaterEqual(nota, Decimal("1.00"))
                    self.assertLessEqual(nota, Decimal("3.00"))


class Faixas(TestCase):
    def test_limites_das_faixas(self):
        self.assertEqual(faixa_da_nota(Decimal("1.70")), "Leve")
        self.assertEqual(faixa_da_nota(Decimal("1.71")), "Moderada")
        self.assertEqual(faixa_da_nota(Decimal("2.30")), "Moderada")
        self.assertEqual(faixa_da_nota(Decimal("2.31")), "Alta")


class Desconto(TestCase):
    def test_sempre_multiplo_de_cinco(self):
        for t in (1, 2, 3):
            for l in (1, 2, 3):
                for e in (1, 2, 3):
                    pct = classificar([Falso(t, l, e)], 100).desconto_percentual
                    self.assertEqual(pct % 5, 0, f"{t}{l}{e} deu {pct}%")

    def test_nunca_passa_do_teto(self):
        for t in (1, 2, 3):
            for l in (1, 2, 3):
                for e in (1, 2, 3):
                    self.assertLessEqual(
                        desconto_da_nota(classificar([Falso(t, l, e)], 100).nota),
                        Decimal("0.50"),
                    )

    def test_nunca_fica_abaixo_do_piso(self):
        self.assertGreaterEqual(desconto_da_nota(Decimal("1.00")), Decimal("0.15"))

    def test_desconto_cresce_com_a_nota(self):
        anterior = Decimal("0")
        notas = sorted(
            {classificar([Falso(t, l, e)], 100).nota
             for t in (1, 2, 3) for l in (1, 2, 3) for e in (1, 2, 3)}
        )
        for nota in notas:
            atual = desconto_da_nota(nota)
            self.assertGreaterEqual(atual, anterior)
            anterior = atual


class MaisDeUmDefeito(TestCase):
    def test_vale_o_mais_grave_e_nao_a_media(self):
        grave = Falso(3, 3, 3)
        leve = Falso(1, 1, 1)
        self.assertEqual(classificar([grave, leve], 200).nota, Decimal("3.00"))

    def test_acrescentar_defeito_leve_nao_melhora_a_peca(self):
        so_grave = classificar([Falso(3, 2, 2)], 200)
        com_leve = classificar([Falso(3, 2, 2), Falso(1, 1, 1)], 200)
        self.assertEqual(so_grave.nota, com_leve.nota)
        self.assertEqual(so_grave.preco_final, com_leve.preco_final)


class PecaSemDefeito(TestCase):
    def test_classificacao_e_nula(self):
        self.assertIsNone(classificar([], 200))
```

```
    def test_preco_final_cai_para_o_original(self):
        peca = Peca.objects.create(
            titulo="Camisa sem defeito", categoria="camisa",
            tamanho="M", preco_original=Decimal("200.00"),
        )
        self.assertIsNone(peca.nota_severidade)
        self.assertIsNone(peca.faixa_severidade)
        self.assertEqual(peca.preco_final, Decimal("200.00"))


class AtravesDoModel(TestCase):
    def setUp(self):
        self.peca = Peca.objects.create(
            titulo="Camisa de linho", categoria="camisa",
            tamanho="M", preco_original=Decimal("289.90"),
        )
        Defeito.objects.create(
            peca=self.peca, tipo=2, localizacao=3, extensao=2,
            descricao="Costura irregular na barra, visível de perto.",
        )

    def test_propriedades_da_peca(self):
        self.assertEqual(self.peca.nota_severidade, Decimal("2.35"))
        self.assertEqual(self.peca.faixa_severidade, "Alta")
        self.assertEqual(self.peca.desconto_percentual, 40)
        self.assertEqual(self.peca.preco_final, Decimal("173.94"))

    def test_nota_do_defeito_isolado(self):
        self.assertEqual(self.peca.defeitos.first().nota, Decimal("2.35"))

    def test_segundo_defeito_mais_grave_muda_o_preco(self):
        Defeito.objects.create(peca=self.peca, tipo=3, localizacao=3, extensao=3)
        self.assertEqual(self.peca.faixa_severidade, "Alta")
        self.assertEqual(self.peca.desconto_percentual, 50)
        self.assertEqual(self.peca.preco_final, Decimal("144.95"))
```